

Chorus
GPUs for ANSI Common Lisp
CUDA, Apple silicon, and AMD
<https://github.com/cgore/chorus/>

Masayuki Takagi
and Christopher Mark Gore
<http://www.cgore.com>
cgore@cgore.com

Monday, October 5th, AD 2026

Contents

Copyright	vii
Introduction	ix
1 The chorus Package	1
1.1 Components	1
2 The chorus/api/context Package	7
2.0.1 The with-cuda Macro	7
2.0.2 The *cuda-device* Variable	7
2.0.3 The *cuda-context* Variable	7
2.0.4 The *cuda-stream* Variable	7
2.0.5 The init-cuda Function	8
2.0.6 The get-cuda-device Function	8
2.0.7 The device-compute-capability Function	8
2.0.8 The create-cuda-context Function	8
2.0.9 The destroy-cuda-context Function	8
2.0.10 The synchronize-context Function	8
3 The chorus/api/memory Package	9
3.1 Memory blocks	9
3.1.1 The alloc-memory-block Function	9
3.1.2 The free-memory-block Function	9
3.1.3 The with-memory-block Macro	9
3.1.4 The with-memory-blocks Macro	9
3.1.5 The sync-memory-block Function	10
3.1.6 The memory-block-aref Function	10
3.2 Separate host and device memory	10
4 The chorus/api/defkernel Package	11
4.0.1 The defkernel Macro	11
4.0.2 The defkernelmacro Macro	11
4.0.3 The defkernel-symbol-macro Macro	11
4.0.4 The defglobal Macro	12
4.0.5 The global-ref Function	12

4.0.6	The <code>expand-macro-1</code> Function	12
4.0.7	The <code>expand-macro</code> Function	12
4.1	The kernel manager	12
5	The <code>chorus/api/macro</code> Package	13
5.0.1	The <code>let*</code> Macro	13
5.0.2	The <code>when</code> Macro	13
5.0.3	The <code>unless</code> Macro	13
6	The <code>chorus/api/nvcc</code> Package	15
6.0.1	The <code>*tmp-path*</code> Variable	15
6.0.2	The <code>*nvcc-options*</code> Variable	15
6.0.3	The <code>*nvcc-binary*</code> Variable	15
6.0.4	The <code>nvcc-arch-option</code> Function	15
6.0.5	The <code>nvcc-available-p</code> Function	16
6.0.6	The <code>find-nvcc</code> Function	16
6.0.7	The <code>find-msvc-cl</code> Function	16
6.0.8	The <code>nvcc-compile</code> Function	16
6.0.9	The <code>arch-option-p</code> Function	16
7	The <code>chorus/api/timer</code> Package	17
7.0.1	The <code>create-timer</code> Function	17
7.0.2	The <code>destroy-timer</code> Function	17
7.0.3	The <code>start-timer</code> Function	17
7.0.4	The <code>stop-timer</code> Function	17
7.0.5	The <code>synchronize-timer</code> Function	17
7.0.6	The <code>elapsed-time</code> Function	18
7.0.7	The <code>with-timer</code> Macro	18
8	The <code>chorus/lang</code> Package	19
8.1	Index variables	19
8.2	Statements	19
8.3	Types	19
8.4	Built-in operations	20
8.5	Compiler entry points	20
9	The <code>chorus/driver-api</code> Package	21
9.1	Variables	21
9.1.1	The <code>*sdk-not-found*</code> Variable	21
9.1.2	The <code>*show-messages*</code> Variable	21
9.2	Types	21
9.3	Event flags	21
9.4	Functions	22
10	The <code>chorus/interop</code> Package	23
10.1	Graphics driver entry points	23
10.2	What the interop API replaces	23

CONTENTS

v

Bibliography

25

Copyright

Copyright © 2012–2021, Masayuki Takagi.

Copyright © 2026, Christopher Mark Gore.

Chorus is licensed under the MIT License. The license text below is the permission notice in the `LICENSE` file.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

`tools/drvapi_error_string.h` is Copyright 1993–2010 NVIDIA Corporation and remains under the terms stated in that file. The MIT terms above apply to the rest of this software.

Introduction

Chorus is a library for GPU programming from Common Lisp. It targets NVIDIA GPUs through CUDA, Apple silicon, and AMD GPUs. It is a fork of `cl-cuda` by Masayuki Takagi [1].

The package `chorus` is the interface most programs use. `*backend*` and `list-devices` select a backend. On a Mac the default is Apple silicon when Metal is available. Where the CUDA driver loads, the default is CUDA. Native CUDA is the package `chorus/cuda`. Native Apple silicon is the package `chorus/apple-silicon`. It keeps a command queue, allocates shared buffers, compiles Metal Shading Language, and launches kernels. Its `defkernel` accepts `let`, `set`, `if`, `aref`, arithmetic, and the index variables. The chapters after this introduction describe the CUDA implementation: the kernel language, the driver API, and `with-cuda`.

The kernel language expresses kernel functions, kernel macros, and kernel symbol macros as S-expressions. On the CUDA backend, a kernel is compiled to CUDA C, then to PTX by `nvcc`, then loaded through the CUDA driver API. When the toolkit is newer than the CUDA version the driver reports, that PTX is rejected and Chorus emits cubin for the selected architecture instead. The kernel manager does this the first time the kernel is launched. A launch looks like an ordinary function call. It takes `:grid-dim` and `:block-dim`, and on the CUDA backend it runs inside a CUDA context.

Memory blocks pair a host allocation with a device allocation. `sync-memory-block` copies between them. The same package also exposes the host pointer and the device pointer separately.

Targets

- NVIDIA GPUs, through CUDA.
- Apple silicon.
- AMD GPUs.

The CUDA backend is verified on Windows and Linux with a current CUDA toolkit. An RTX 50-series GPU (Blackwell, compute capability 12.0) needs CUDA Toolkit 12.8 or later. CUDA 13.x is the toolkit this backend is written for. With CUDA 13, `nvcc` can target Turing and newer (`sm_75` and above).

The repository is at <https://github.com/cgore/chorus/>.

Requirements for the CUDA backend

- An NVIDIA CUDA-capable GPU.
- The CUDA driver (`nvcuda.dll` on Windows, `libcuda.so.1` on Linux).
- The CUDA Toolkit, so that `nvcc` can compile kernels to PTX.
- On Windows, Visual Studio with the C++ workload, so that `nvcc` has a host compiler.

Kernel files are written to the operating system's temporary directory unless `*tmp-path*` is set. `nvcc` is found on `PATH`, through `CUDA_PATH` or `CUDA_HOME`, or in the usual toolkit locations.

Loading the library

Once the checkout is visible to ASDF:

```
(ql:quickload :chorus)
```

The test suite runs when `chorus-test` is loaded:

```
(asdf:test-system :chorus)
```

From a shell, in the checkout:

```
sbcl --load ~/quicklisp/setup.lisp --load test/run.lisp
```

A kernel

The following is the CUDA backend's form of `examples/vector-add.lisp`.

```
(defkernel vec-add-kernel (void ((a float*) (b float*) (c float*) (n int)))
  (let ((i (+ (* block-dim-x block-idx-x) thread-idx-x)))
    (if (< i n)
        (set (aref c i)
              (+ (aref a i) (aref b i)))))
  (defun main ()
```

```

(let* ((dev-id 0)
      (n 1024)
      (threads-per-block 256)
      (blocks-per-grid (/ n threads-per-block)))
  (with-cuda (dev-id)
    (with-memory-blocks ((a 'float n)
                        (b 'float n)
                        (c 'float n))
      (random-init a n)
      (random-init b n)
      (sync-memory-block a :host-to-device)
      (sync-memory-block b :host-to-device)
      (vec-add-kernel a b c n
        :grid-dim (list blocks-per-grid 1 1)
        :block-dim (list threads-per-block 1 1))
      (sync-memory-block c :device-to-host)
      (verify-result a b c n))))

```

`aref` reads an array element. `set` writes one. `block-dim-x`, `block-idx-x`, and `thread-idx-x` are the kernel language's index variables. The CUDA backend maps them to the CUDA built-ins.

Chapter 1

The chorus Package

Load this package with `(ql:quickload :chorus)`. It reexports the generic backend interface from `chorus/backend`, the kernel language from `chorus/lang`, the CUDA API from `chorus/api`, and two driver variables from `chorus/driver-api`: `*show-messages*` and `*sdk-not-found*`.

`*backend*` and `list-devices` are the calls most programs make. `chorus/cuda` is the native CUDA package. `chorus/apple-silicon` is the native Apple silicon package.

`*sdk-not-found*` is true when the CUDA driver library does not load. Driver calls then signal `sdk-not-found-error`. Chorus also targets Apple silicon and AMD GPUs.

The packages below are the ones `chorus` is built from. Use them when you want a narrower set of symbols than the reexport.

1.1 Components

The system file `chorus.asd` defines one module, named `source`. Each Lisp file in that module is a component. The component's name is its path under `source`, without the type. ASDF resolves a `:depends-on` name among those components.

A component depends on another when its file uses a definition from that file. The use may be a package the file enters or reexports, a macro the file expands, a foreign type the file names, or a function, variable, or constant the file refers to. The figures draw those dependencies as trees. A file stands to the left of the files it depends on. An arrow points at the file loaded first. A file that several others depend on appears once beside each of them. A shaded box is a file of the group in the caption. A white box is a file from another directory, included so the arrow has a target.

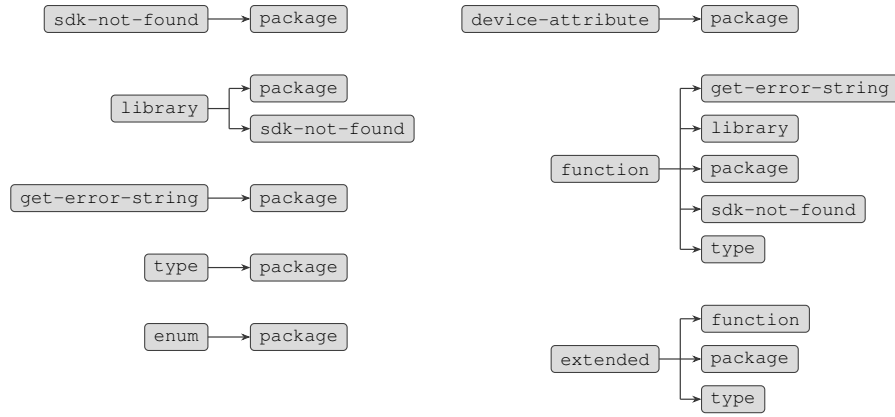


Figure 1.1: Driver API components, the files under `source/driver-api/`. `function` depends on `library` because `defcufun` reads `*sdk-not-found*` when it expands, and `library` sets that variable.

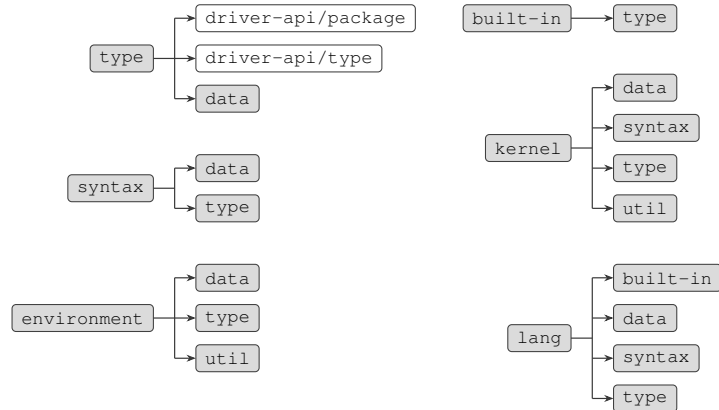


Figure 1.2: Kernel language components, the files under `source/lang/` other than the compiler. The white boxes are the driver files that `type` uses.

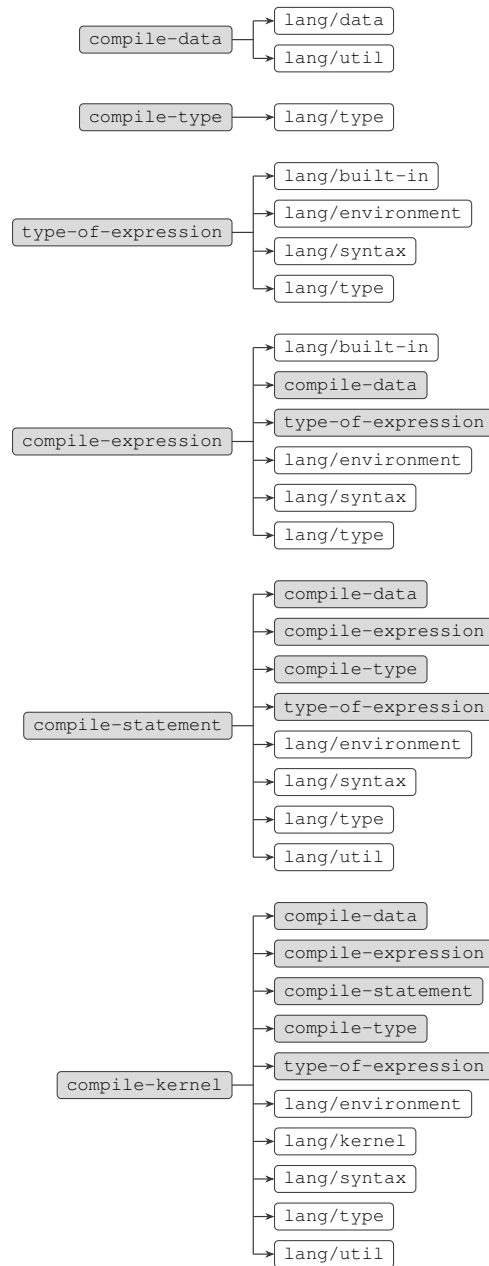


Figure 1.3: Compiler components, the files under `source/lang/compiler/`. The white boxes are the language files the compiler uses.

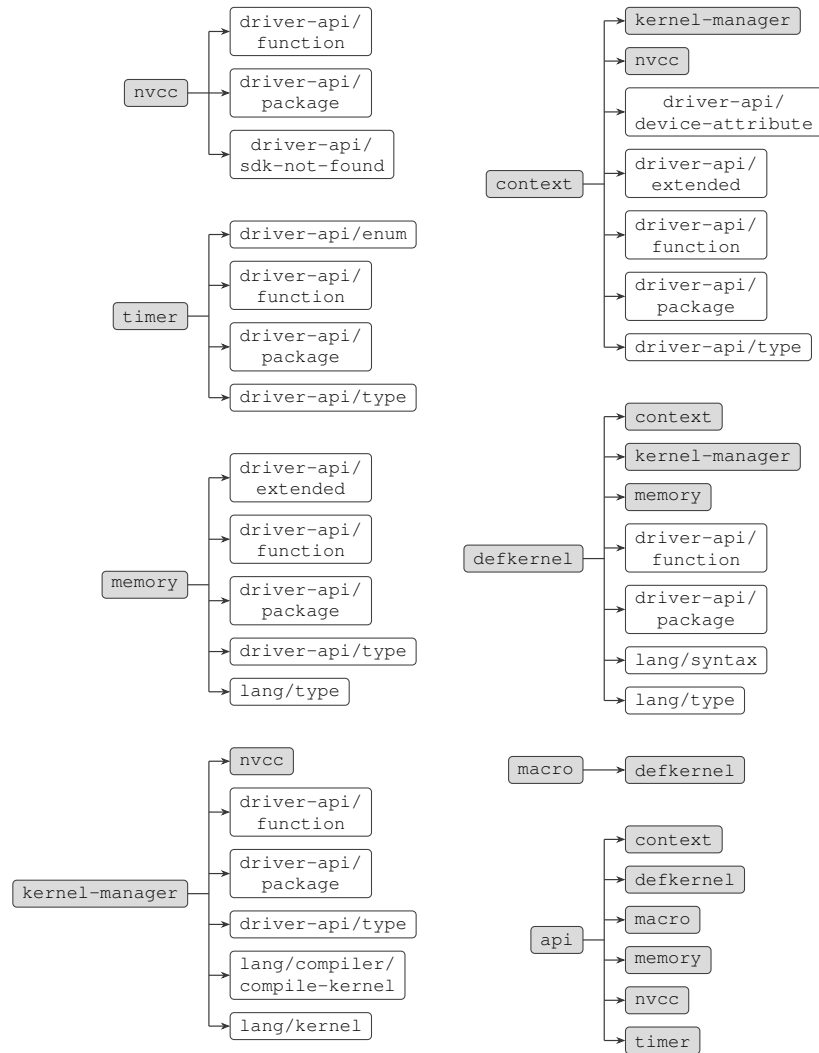


Figure 1.4: CUDA API components, the shaded boxes, under `source/api/`. A name that includes a directory is a file outside that directory.

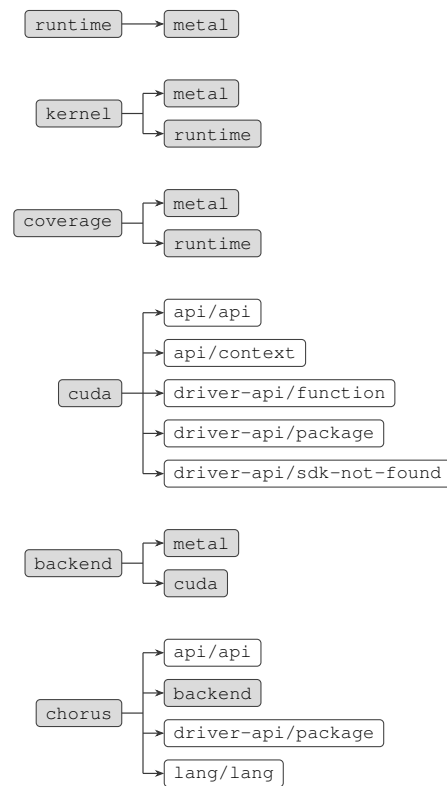


Figure 1.5: Apple silicon, cuda, backend, and chorus. backend depends on apple-silicon/metal and cuda.

Chapter 2

The chorus/api/context Package

This package initializes CUDA, selects a device, and keeps a context. `chorus/api` reexports it.

2.0.1 The `with-cuda` Macro

Syntax

```
(with-cuda (dev-id) &body body)
```

The macro calls `init-cuda`, binds `*cuda-device*` from `dev-id`, binds `*cuda-context*`, and runs `body`. If `*nvcc-options*` does not already name an architecture, the macro adds the `nvcc -arch=sm.XY` option for that device's compute capability. On the way out it unloads `*kernel-manager*` and destroys the context.

2.0.2 The `*cuda-device*` Variable

The device selected by `with-cuda`.

2.0.3 The `*cuda-context*` Variable

The context created by `with-cuda`.

2.0.4 The `*cuda-stream*` Variable

The stream passed to kernel launches. Its initial value is a null pointer.

2.0.5 The `init-cuda` Function

Calls `cu-init` with flags 0.

2.0.6 The `get-cuda-device` Function

Returns the device for an ordinal.

2.0.7 The `device-compute-capability` Function

Returns the compute capability of a device.

2.0.8 The `create-cuda-context` Function

Creates a context on a device.

2.0.9 The `destroy-cuda-context` Function

Destroys a context.

2.0.10 The `synchronize-context` Function

Waits until the work in the context has finished.

Chapter 3

The chorus/api/memory Package

This package allocates device memory, host memory, and memory blocks that pair the two. `chorus/api` reexports it.

3.1 Memory blocks

3.1.1 The `alloc-memory-block` Function

Syntax

```
(alloc-memory-block type size)
```

Allocates a block of *size* elements of *type*.

3.1.2 The `free-memory-block` Function

Frees both the device pointer and the host pointer of a block.

3.1.3 The `with-memory-block` Macro

Syntax

```
(with-memory-block (var type size) &body body)
```

Binds *var* to a new block and frees it on the way out.

3.1.4 The `with-memory-blocks` Macro

Syntax

```
(with-memory-blocks bindings &body body)
```

Each binding has the form `(var type size)`. The macro nests `with-memory-block`.

3.1.5 The sync-memory-block Function

Syntax

```
(sync-memory-block memory-block direction)
  direction is :host-to-device or :device-to-host.
```

3.1.6 The memory-block-aref Function

Syntax

```
(memory-block-aref memory-block index)
```

Reads the host side of the block. `setf` writes it. A copy to or from the device is a separate `sync-memory-block` call.

The package also exports `memory-block-p`, `memory-block-device-ptr`, `memory-block-host-ptr`, `memory-block-type`, and `memory-block-size`.

3.2 Separate host and device memory

`alloc-device-memory` takes a type and a count. `free-device-memory` frees that pointer. `with-device-memory` has the form `(with-device-memory (var type n) &body body)`.

`alloc-host-memory`, `free-host-memory`, and `host-memory-aref` are the host-side counterparts. `with-host-memory` has the same form as `with-device-memory`.

`memcpy-host-to-device` and `memcpy-device-to-host` copy a region. `device-total-bytes`, `device-total-kbytes`, `device-total-mbytes`, and `device-total-gbytes` report the memory on a device. `device-total-bytes` takes the device as its argument.

Chapter 4

The chorus/api/defkernel Package

This package defines kernels and launches them. `chorus/api` reexports it. It does not reexport `chorus/api/kernel-manager`. A kernel defined here is registered on `*kernel-manager*` and compiled the first time it is launched.

4.0.1 The `defkernel` Macro

Syntax

```
(defkernel name (return-type arguments) &body body)
```

Each argument has the form `(var type)`. The macro defines a Lisp function of those variables with keyword arguments `grid-dim` and `block-dim`. Both default to `(1 1 1)`. The launch uses `cu-launch-kernel` and the stream in `chorus/api/context: *cuda-stream*`.

4.0.2 The `defkernelmacro` Macro

Syntax

```
(defkernelmacro name arguments &body body)
```

Defines a macro in the kernel language.

4.0.3 The `defkernel-symbol-macro` Macro

Syntax

```
(defkernel-symbol-macro name expansion)
```

Defines a symbol macro in the kernel language.

4.0.4 The defglobal Macro

Syntax

```
(defglobal name expression &optional qualifiers)
```

Defines a kernel global. When *qualifiers* is omitted, the qualifier is `:device`.

4.0.5 The global-ref Function

Returns the device pointer of a global after ensuring it is loaded.

4.0.6 The expand-macro-1 Function

Expands one kernel-macro call.

4.0.7 The expand-macro Function

Expands kernel macros in a form.

4.1 The kernel manager

`chorus/api/kernel-manager` holds the definitions and the loaded CUDA module. `*kernel-manager*` is that object. `defkernel` calls `kernel-manager-define-function`. A `launch` calls `ensure-kernel-function-loaded`. `with-cuda` calls `kernel-manager-unload` when the context ends.

The package also exports `make-kernel-manager`, the `kernel-manager-*` readers, `kernel-manager-compile-module`, `kernel-manager-load-module`, `kernel-manager-load-function`, `kernel-manager-load-global`, and the other `ensure-kernel-*` operations.

Chapter 5

The chorus/api/macro Package

These are kernel macros, defined with `defkernelmacro`. `chorus/api` reexports them. A kernel that uses them should import or use this package. The test `chorus/test/api/smoke` shadows imports `let*`, `when`, and `unless` from here so that the kernel body sees the kernel versions.

5.0.1 The `let*` Macro

Binds sequentially. An empty binding list expands to `progn`.

Syntax

```
(let* bindings &body body)
```

5.0.2 The `when` Macro

Syntax

```
(when test &body body)  
Expands to (if test (progn body)).
```

5.0.3 The `unless` Macro

Syntax

```
(unless test &body body)  
Expands to (if (not test) (progn body)).
```


Chapter 6

The chorus/api/nvcc Package

This package finds `nvcc` and invokes it. `chorus/api` reexports `*tmp-path*`, `*nvcc-options*`, `*nvcc-binary*`, `find-nvcc`, `nvcc-available-p`, and `nvcc-arch-option`. `nvcc-compile`, `find-msvc-cl`, and `arch-option-p` stay in this package.

6.0.1 The `*tmp-path*` Variable

Directory for temporary `.cu` and `.ptx` files. `nil` means `uiop:temporary-directory`.

6.0.2 The `*nvcc-options*` Variable

Extra command-line options passed to `nvcc`.

6.0.3 The `*nvcc-binary*` Variable

Path to `nvcc`. `nil` or the string `nvcc` means auto-detect, using `PATH`, `CUDA_PATH`, and the usual toolkit locations on Windows and Unix.

6.0.4 The `nvcc-arch-option` Function

Syntax

`(nvcc-arch-option major minor)`

Returns an `-arch=sm_XY` option. Compute capability 12.0 becomes `sm_120`. `major` is a nonnegative integer. `minor` is an integer from 0 through 9.

6.0.5 The `nvcc-available-p` Function

True when an `nvcc` binary can be found.

6.0.6 The `find-nvcc` Function

Returns the `nvcc` pathname that auto-detect selected.

6.0.7 The `find-msvc-cl` Function

Returns the Windows host compiler `cl.exe` when one is found.

6.0.8 The `nvcc-compile` Function

Compiles a CUDA C string to PTX. The include path passed to `nvcc` is the `include/` directory of the `chorus` system.

6.0.9 The `arch-option-p` Function

True when an option list already contains an architecture selector.

Chapter 7

The chorus/api/timer Package

Timers are built from CUDA events. `chorus/api` reexports this package.

7.0.1 The `create-timer` Function

Syntax

```
(create-timer)
```

7.0.2 The `destroy-timer` Function

Destroys a timer.

7.0.3 The `start-timer` Function

Records the start event.

7.0.4 The `stop-timer` Function

Records the stop event.

7.0.5 The `synchronize-timer` Function

Waits for the timer's events.

7.0.6 The elapsed-time Function

Syntax

```
(elapsed-time timer)
```

7.0.7 The with-timer Macro

Syntax

```
(with-timer (var) &body body)
```

Binds *var* to a new timer and destroys it on the way out.

Chapter 8

The chorus/lang Package

This is the kernel language. `chorus` reexports it. Every generated `.cu` file includes `int.h`, `float.h`, `float3.h`, `float4.h`, `double.h`, `double3.h`, `double4.h`, and `curand.h` from the `include/` directory.

8.1 Index variables

`grid-dim-x`, `grid-dim-y`, `grid-dim-z`, `block-dim-x`, `block-dim-y`, `block-dim-z`, `block-idx-x`, `block-idx-y`, `block-idx-z`, `thread-idx-x`, `thread-idx-y`, and `thread-idx-z`.

8.2 Statements

`set` stores a value. `with-shared-memory` declares shared memory for the block.

8.3 Types

Scalar types: `void`, `bool`, `int`, `float`, `double`, and `curand-state-xorwow`.

Vector types, also available as Lisp structures: `float3`, `float4`, `double3`, and `double4`. Each has a constructor `make-float3` (and the same pattern for the others), slot readers, a predicate, an equality test, and a `with-float3` macro.

Pointer types: `bool*`, `int*`, `float*`, `double*`, `float3*`, `float4*`, `double3*`, `double4*`, and `curand-state-xorwow*`.

`cffi-type` maps a Chorus type to the CFFI type used at a launch.

8.4 Built-in operations

xor, shl, shr, rsqrt, __exp, __divide, atomic-add, pointer, syncthread, double-to-int-rn, dot, curand-init-xorwow, curand-uniform-float-xorwow, curand-uniform-double-xorwow, curand-normal-float-xorwow, and curand-normal-double-xorwow.

8.5 Compiler entry points

The compiler packages export one operation each: `compile-symbol`, `compile-type`, `type-of-expression`, `compile-expression`, `compile-statement`, and `compile-kernel`. `compile-kernel` is what the kernel manager calls.

Chapter 9

The chorus/driver-api Package

This package is the foreign binding to the CUDA driver API. Names follow the driver entry points with a `cu-` prefix and Lisp hyphens. When the driver library is absent, a call signals `sdk-not-found-error`.

9.1 Variables

9.1.1 The `*sdk-not-found*` Variable

True when the driver library did not load.

9.1.2 The `*show-messages*` Variable

Controls driver-API diagnostic messages. `chorus` reexports both variables.

9.2 Types

`cu-result`, `cu-device`, `cu-context`, `cu-module`, `cu-function`, `cu-stream`, `cu-event`, `cu-device-ptr`, and `size-t`.

9.3 Event flags

`cu-event-default`, `cu-event-blocking-sync`, `cu-event-disable-timing`, and `cu-event-interprocess`.

9.4 Functions

Device and context: `cu-init`, `cu-device-get`, `cu-device-get-count`, `cu-device-compute-capability`, `cu-device-get-attribute`, `cu-device-attribute-compute-capability`, `cu-device-attribute-compute-capability-minor`, `cu-device-get-name`, `cu-ctx-create`, `cu-ctx-destroy`, `cu-ctx-synchronize`, **and** `cu-device-total-mem`.

Memory: `cu-mem-alloc`, `cu-mem-free`, `cu-mem-host-register`, `cu-mem-host-unregister`, `cu-memcpy-host-to-device`, `cu-memcpy-host-to-device-async`, `cu-memcpy-device-to-host`, **and** `cu-memcpy-device-to-host-async`.

Modules and launch: `cu-module-load`, `cu-module-unload`, `cu-module-get-function`, `cu-module-get-global`, **and** `cu-launch-kernel`.

Events: `cu-event-create`, `cu-event-destroy`, `cu-event-elapsed-time`, `cu-event-record`, **and** `cu-event-synchronize`.

Streams: `cu-stream-create`, `cu-stream-destroy`, `cu-stream-query`, `cu-stream-synchronize`, **and** `cu-stream-wait-event`.

Chapter 10

The chorus/interop Package

This package is a separate system, `chorus-interop`. It depends on `chorus`, `cl-opengl`, `cl-glu`, and `cl-glut`. Load it with `(ql:quickload :chorus-interop)`.

It reexports `chorus/lang`, the interop API, and the driver variables `*show-messages*` and `*sdk-not-found*`.

10.1 Graphics driver entry points

`chorus/interop/driver-api` adds `cu-graphics-resource`, the `cu-graphics-register-flags-*` and `cu-graphics-map-resource-flags-*` enumerations, and these functions: `cu-gl-ctx-create`, `cu-graphics-gl-register-buffer`, `cu-graphics-map-resources`, `cu-graphics-resource-get-mapped-pointer`, `cu-graphics-resource-set-map-flags`, `cu-graphics-unmap-resources`, and `cu-graphics-unregister-resource`. It also reexports `chorus/driver-api`.

10.2 What the interop API replaces

`chorus/interop/api` reexports `chorus/api` except the `context`, `memory-block`, and `defkernel` symbols that the interop packages replace. The replacements are `create-cuda-context`, `with-cuda`, `defkernel`, and the memory-block operations, including `memory-block-vertex-buffer-object` and `memory-block-graphic-resource-ptr`.

The example `interop/examples/nbody.lisp` is the system `chorus-interop-examples`. Its package is `chorus/interop/examples/nbody`.

Bibliography

- [1] TAKAGI, M. cl-cuda. <https://github.com/takagi/cl-cuda>, 2012.